

PRD — Actual Production Capture & Stock Ledger (SM Back-Office + Crew KDS)

Portfolio restructure — combines PRD 3.1 (Actual Production) and PRD 3.6 (Crew Actual on KDS), reunited as one capture-and-ledger spec spanning both surfaces it always governed.

- **Status:** Merged for portfolio — the KDS surface was carved out of this scope on 03 Aug so the crew-facing detail (ticket rendering, offline queue-and-replay) could be spec'd on its own; reunited here since both are one contract with two front ends, not two products.
- **Owner:** Jauhar Nurandyo
- **Epic:** Project Brief - EPIC 3 - Production Planning
- **Phase:** P1 (SM back-office form, next sprint) + P2 (crew KDS, built to the POS-team spec, on Dash POS)
- **Tenant scope:** the client — QSR chain
- **Model:** exit-only deduction (EPIC 2) unchanged — stock deducts at sale and waste only; production, on either surface, is a pure stock-in and deducts nothing.
- **Surface:** two first-class surfaces, both posting the identical stock-in — the SM “Input Actual Cook” form (back-office) and the crew/SM KDS (Dash POS tablet). Correction lives only at the back office (SM/MOD); the KDS has none.

1. Objective

Give the outlet a way to record what it cooked, on whichever surface fits the moment. The operator — SM at a desk, or crew at the station — records what was actually produced and confirms each task as it is made. A confirmation banks the actual quantity of that item (FG or SFG) into the outlet's stock and deducts nothing; consumption is recognized later, at the exits (sale and waste), by the EPIC 2 explosion. This is the only manual stock-in in the production loop, and the load-bearing spec of EPIC 3: nothing downstream (plan-vs-actual, waste exit, DOI) has stock to work with until it ships.

This document specifies the full loop: the capture model, the stock-in movement a confirmation writes, the used / outstanding / current ledger it computes for others to read (the engine behind the planning flow's effectively available figure), and both surfaces that write against it. It does not specify planning itself, the EPIC 2 deduction engine, or waste (EPIC 4).

Scope boundary. Production is confirmed against the day's Production Order, at the plan line's cycle — Hourly in pieces with fryer/temperature, Daypart in grams with a batch count, both with a ready time. The operator records what was genuinely produced and confirms the task; the plan is shown as a pre-filled, editable target, never a value that posts itself. The actual may be above, below, or on plan. It posts stock-in only; deduction is the EPIC 2 exit-only engine. FG and SFG are both produced and confirmed here — SFG is a stocked, plannable cooked intermediate. Stock coverage is enforced at the planning phase (the planning flow's Raw Material shortfall step), not at this entry — production confirmation is never stock-gated.

2. Problem Statements

- The EPIC 2 engine deliberately never increments stock — there's no way to bank a cooked FG or prepped SFG into outlet inventory, and the exit-side deduction has nothing to deduct against.

- Under exit-only, the raw book overstates intraday by the recipe content locked inside cooked-but-unexited goods. The planning preview and the DOI signal need an honest effectively available read — the used / outstanding / current computation this spec owns.
- Cooking happens on two different rhythms: the desk-based SM reconciling at end of shift, and the crew at the station under rush, on a tablet. A single surface that assumes typing, shows stock, or assumes real-time desk entry will be worked around or filled in badly on one side or the other — and the ledger inherits that.
- Kitchen wifi is unreliable. A confirmation tapped during a drop must post exactly once on reconnect, and must keep the real cook time — never the sync time.

3. Goals

- A task-level production confirmation, on either surface: confirming a task posts an immediate stock-in of the item — FG or SFG — at the actual quantity produced, deducting nothing. The plan never forces the actual.
- Capture the actual production, defaulting to one summed quantity per task but allowing separate batch entries where the cook genuinely happened in batches, each with its own ready time and shelf-life clock.
- Define and compute the used / outstanding / current ledger per item — an engine, not a crew-facing surface — yielding effectively available = current – outstanding.
- Capture produced_at and ready time distinct from confirm time, so end-of-shift confirmation is first-class and EPIC 4's shelf-life clock starts from the right moment.
- Ship both surfaces as first-class: the SM entry form (P1) and the crew KDS (P2) — posting the identical stock-in contract.
- A KDS capture sheet that asks only what fits the item, pre-fills the plan, and confirms in one primary action — target under ten seconds for an on-plan cook.
- Offline queue-and-replay on the KDS that never double-posts and preserves produced_at.
- A correction path — same-day in-place edit; void for older — gated to an authorized back-office role and audited. The KDS exposes no correction and no stock figure anywhere.

4. The Used / Outstanding / Current Ledger

Defined and computed here — an engine, not a crew surface: the crew never see it, and the outlet UI shows at most a single derived figure. Exit-only's known cost: between cook and exit, the book still counts material physically inside the warmer or prep tub. This engine corrects that in the read layer, never the ledger.

Per item (RM, and SFG as a stocked tier):

- **Current** — the book balance as posted (stock-ins minus exit deductions).
- **Outstanding** — the recipe content committed inside cooked-but-unexited goods, derived from production confirmations minus exits, exploded through the BOM. Two tiers, since SFG is stocked: an unexited FG commits its SFG and direct RM; an unexited SFG commits its RM.
- **Used** — consumed by exits (sale/waste explosions); shown for completeness.
- **Effectively available = current – outstanding** — the honest coverage figure.

Rules:

- Derived at read time; never stored as movements — persisting committed quantities as movements would be production-time deduction re-entering by the side door, a reversal of the exit-only decision.
- Computed with the same BOM-explosion code path and yield formula the engine uses at exit, so the outstanding estimate and the eventual deduction agree.
- PM (packaging) never appears outstanding — it deducts flat at SALE only.
- An estimate against the current BOM — a mid-shelf-life BOM edit skews it slightly; acceptable, labeled where surfaced.
- Consumers: the planning flow's per-plan Raw Material shortfall check, and EPIC 5's stock-status/DOI signal, which should read effectively available rather than raw.

5. System Flow & Behavior

5.1 The three surfaces — Order, Task, KDS

A submitted plan creates its Production Order; production is confirmed against its tasks. The same objects appear at three zoom levels:

Surface	Scope	Audience
Production Order	One per submitted plan (e.g. PO-2601, “15 tasks · 324 pcs”) — one rollup status and progress bar. Created by the planning flow at submit.	SM tracking “is this order on track?”
Production Task	One per item × slot (e.g. TSK-2601-1, Crispy Chicken Piece, Breakfast 07:00, 52 pcs) — own status; flat list with station + status filters.	Dispatch/floor: every pending task at a station now.
KDS	One station, now — tasks as live tickets in Pending / In Progress / Done.	The crew at that station.

Ownership. The planning flow creates the Order and Tasks at submit; the Production Order & Task Contract owns their entities, state machine, and dispatch/feed contract. This document owns exactly one entity — `production_entry` (§6) — and specifies the task's capture lifecycle and what a confirmation writes: the stock-in contract that binds both the SM form and the KDS.

Order → Task is created at submit: the plan explodes into tasks, one per item×slot line, each tagged item / slot / quantity / station. Order = the folder; Task = the file; KDS = one station's files for the current window. FG and SFG tasks alike.

The Production Order also carries, when EPIC 4 lands, REWORK and DISPOSE job lines on cooked output at close, and leftover SFG (all SFG expire by EOD) exits via WASTE there. Those lines never render before EPIC 4 ships. The crew's surface stays execute-only — what to cook, when — and never shows a stock readout.

5.2 The confirmation — task-level, cycle-aware, actual-capturing

A confirmation covers one Production Task — an FG or SFG: sellable cooked output (modifiers included) or a stocked cooked intermediate (dough, sauces, premixes prepped at open). The task shapes to the plan line's cycle: an Hourly task shows pieces with fryer (A–D) and temperature; a Daypart task shows grams with a batch count for the window; both with a ready time. The unit of confirmation is the task (one item × slot) — components are never entered or deducted here.

The operator records the actual produced quantity, not the plan. The task shows the planned quantity pre-filled and editable; what posts is what the operator enters — more, less, or equal. The plan is a target, never a value that posts itself.

One summed actual by default; separate batches where cooked that way. Where the cook genuinely happened in multiple runs (20 at 06:00, 30 at 09:00, 50 at 11:00), each may be recorded as its own batch entry, each carrying its own quantity, ready time, and produced_at. One summed entry and N batch entries are both accepted; the task's actual is the sum of its entries.

Cook-context fields captured per entry:

Field	Required	Notes
Outlet (branch)	Yes	The producing outlet
Task / produced item	Yes	product_variant.id, FG or SFG — never Menu, RM, PM, SS
Actual quantity	Yes	What was actually cooked, at the cycle's unit — Hourly: pieces; Daypart: grams. Planned qty pre-filled and editable
Batch count	Daypart only	Number of BOM cook-runs this entry represents
Production code	Yes	Recorded on the cook
Fryer	Hourly, optional	A–D — which fryer cooked the run
Temperature	Hourly, optional	The cook temperature
Ready time	Yes	When ready — EPIC 4's shelf-life clock starts here, per entry
produced_at	Yes	When the cook physically happened; defaults to now, backdatable ≤3h same business day
Note	Optional	Free text

Yield is not entered here — the BOM's yield/scrap ratio applies at explosion time; the operator confirms output, not consumption.

5.3 The task lifecycle — Pending → In Progress → Done

Tasks release on a clock-driven cadence; the plan's cycle is the release clock.

- **Pending** — releases 1 hour before slot start (e.g. a Lunch/11:00 task at 10:00). Before that it isn't on the floor.
- **In Progress** — the slot's timeframe is current; the task is live and cookable, and counts toward the Production Order's progress bar. Confirmation is available here, including cooked-ahead.
- **Done** — the operator confirms; the task rolls up into the Order's progress. The confirmation that moves a task to Done posts the stock-in (§5.4).
- Daypart = same machinery, longer window — only the timeframe stretches. FG and SFG alike.

Station routing. A task routes to exactly one station (Fryer Line, Grill, Prep/Sides, Bar, Assembler) via the item's station_id — reserved, not set in V1 (master owned by another PM). Until set, tasks fall back to a single/undifferentiated view or a provisional tag.

5.4 The confirmation posts the actual cook as stock-in

The entered actual quantity is what posts, not the plan. Confirming posts PRODUCTION_STOCK_IN for the item — FG or SFG — at the actual quantity, at the outlet, immediately. Where multiple batches were logged, each posts its own stock-in on its own confirmation.

What a confirmation writes, in sequence — identical whichever surface captured it. On Save the system:

- 1. Validates: item is FG or SFG, actual quantity > 0, ready time present, produced_at within the ≤3h same-business-day backdate window — no stock check of any kind.
- 2. Writes one production_entry row (§6) with the actual quantity, cook context, ready time, produced_at, and entered_via (SM_FORM or KDS).
- 3. Posts PRODUCTION_STOCK_IN for product_variant_id × outlet at the entered actual quantity, onto the outlet's own stock (qsr-inventory, per-shop) — deducting nothing, no BOM explosion here.
- 4. Advances the task to Done, recomputing the Order's rollup.

Two rules bind this on every surface:

- **One entry = one movement** — the entry id is the idempotency key, so a re-tapped or retrieved Save never posts twice.
- **produced_at, not save or sync time, starts EPIC 4's shelf-life clock.**

Every movement carries operation_id, trigger type, entry id, operator, and surface, so a stock figure traces back to the cook that made it.

Production confirmation is never stock-gated. The stock-coverage concern is enforced at the planning phase, where the planning flow's Raw Material step shows the shortfall and routes the SM to a Goods Receipt / transfer ahead of the cook day. If raw goes momentarily negative, EPIC 5's stock-status signal surfaces it to the SM as a replenishment matter. Sales are never blocked.

The Production Order carries only items planned above zero — a skipped item has no COOK task by design, so there's nothing to confirm for it. Because the posted quantity is the recorded actual and not the plan, plan-vs-actual (reports) is a genuine figure.

5.5 Two surfaces, one contract

Actual production is captured on two surfaces, both first-class in the end state. Invariants, binding on both:

- Every confirmation captures a real actual — the plan is a pre-filled, editable default, never an assumed post.
- One task → 1..N entries; each batch carries its own ready time and produced_at.
- Both surfaces post the identical PRODUCTION_STOCK_IN through §5.4 — same validation, same idempotency, same clock rule.
- Neither blocks; sales are never affected. Divergence (cooked ≠ planned) is editing the quantity, not a separate mode.

Phasing: Phase 1 (next sprint) — the SM form is the sole capture path. Phase 2 (when the KDS ships) — the crew (and SM) input on the KDS as the primary surface, and the SM form remains available, never retired. The phases change which surface is primary, never whether the SM path exists.

5.5a The SM form — “Input Actual Cook” (back-office)

Built for a desk, often used after the fact — end of shift, reconciliation. Typed fields and dropdowns, not touch tiles.

Field	State	Note
Task reference	read-only	e.g. TSK-2602-12
Branch/Outlet · Planning Date · Product · Cooking Time	read-only	Context from the task. Label reads Branch/Outlet, never “Plant”
Planning Cook (unit)	read-only	The plan, for reference
Actual Cook (unit)	Yes	What was actually cooked — the figure that posts
Ready Time (hour · minute)	Yes	Captured per entry
Production Code	Yes	Recorded on the cook
End Shelf Life	read-only	From product master, HO-set — display only
Note	Optional	Free text
Batal / Simpan	—	Simpan posts the stock-in

The SM may record several batch entries against one task, or one summed entry; the task's actual is the sum.

5.5b The crew KDS (Dash POS)

One station at a time (Frying · Prep · Beverage · Sauce), switchable at the top, alongside outlet, clock, and operating-hour rail. Renders released tasks for that station — task_id, item, slot, planned qty, station, state — ordered by slot. Release timing and state are computed server-side by the Production Order & Task Contract; the KDS renders states, it never computes them.

Tickets show three states: Pending (released, not started), In Progress (slot current, cookable/confirmable), Done (confirmed). An overdue task (slot passed, unconfirmed) stays In Progress with a visible overdue flag — never auto-completed.

The capture sheet — the only place a human records reality:

#	Block	Behavior
1	Identity	Autofilled: id, item name, planned qty, slot, station, Production Code. Zero typing
2	Actual Cooked	The one real input — a large surface opening a large numpad, pre-filled with planned qty (one tap = cooked as planned; edit to diverge)
3	Fryer & Temperature	Frying-station only: Fryer A–D toggles; active fryer reveals a temperature stepper (default 170°C)
4	Timing	Cooked at (default just now, ±15m steps, backdatable ≤3h) · Ready time · Fresh until (derived, read-only: ready + item's hold duration)
5	Bump	Full-width primary — advances task to Done, ticks Order progress, posts the stock-in; brief success toast, card collapses

The sheet shapes to the plan line's cycle, same as §5.2. Yield is never entered.

What the KDS never does: no edit, no void, no correction of any kind. No stock readout. No plan-vs-actual, no reports.

Offline capture: if the tablet drops connection, Bumps queue locally and replay on reconnect — each posts exactly once (entry id as idempotency key) and preserves its produced_at.

5.6 Correction — same-day in-place edit; void for older, back-office only

An entry confirmed with a wrong quantity or cook context is corrected, same business day, by editing its fields in place: the posted stock-in adjusts to the corrected values. The edit is stamped (who, when) and retained for audit. Gated to an authorized outlet role (provisionally SM/MOD) — crew confirm, an authorized role corrects; the KDS has no correction UI at all.

Beyond the same-day window, or for a full reversal, the entry is voided: a reversing PRODUCTION_VOID backs out the original stock-in only — there's no consumption leg to unwind, since production never deducted. A void is not a WASTE exit and not a silent edit; both the original and reversal remain in the audit trail. Entries older than the window route to the adjustment primitive (open dependency). To re-confirm after a void, the operator enters a fresh confirmation.

6. Data Model (additive)

New entity: production_entry (QSR module). One row per confirmed entry — one per task by default, or one per batch where the cook was split. No new entity is introduced by the KDS surface; it writes into this same table.

Column	Type	Notes
id	uuid	PK
company_id	uuid	Tenant
branch_id	uuid	The producing outlet
production_task_id	uuid	The Production Task confirmed
product_variant_id	uuid	The produced item — FG or SFG
quantity	numeric	The actual produced quantity, cycle's unit
is_batch_split	bool	True where one of several batches against a task
entered_via	enum	SM_FORM or KDS — which surface captured the entry
batch_count	int, nullable	Daypart only
fryer	text, nullable	Hourly only (A–D)
temperature	numeric, nullable	Hourly only
ready_time	timestampz	EPIC 4's clock start
produced_at	timestampz	Backdatable ≤3h same business day
status	enum	posted / voided
note	text, nullable	Free text
audit set	—	created_by/at, corrected_by/at, voided_by/at

Movements written: PRODUCTION_STOCK_IN on confirm, PRODUCTION_VOID on void — uniform audit shape (operation_id, trigger_event_type, trigger_event_id). The used/outstanding/current read (\$4) writes no movements.

7. User Stories & Acceptance Criteria

Story 1 — Production tasks surface and move through their lifecycle

User Story

As the outlet, the day's tasks appear as their time approaches and move Pending → In Progress → Done as they're worked, so what to cook next is always visible and stock reflects what was produced.

Acceptance Criteria

- A task releases Pending exactly 1 hour before slot start; before that it isn't on the floor.
- A task moves to In Progress when its slot becomes current; becomes cookable and counts toward the Order's progress bar.
- Confirmation is available in In Progress, including early when already all cooked.
- Confirming moves it to Done, rolls up progress, posts PRODUCTION_STOCK_IN at the actual quantity entered.
- Never blocked for stock; sales never blocked. Daypart and Hourly share the identical lifecycle.
- A task routes to exactly one station via station_id (reserved, not set in V1).
- The floor sees tasks and states only — never a stock readout.

AC Scenarios (Gherkin)

Scenario: A Lunch task's full lifecycle

```
Given TSK-2601-2 (Crispy Chicken Piece, Lunch 11:00, planned 72) on the Fryer Line
When the clock reaches 10:00
Then the task releases as Pending
When the clock reaches 11:00
Then the task turns In Progress and counts toward PO-2601's progress bar
When it is confirmed after cooking
Then it moves to Done, rolls up to PO-2601's progress
And PRODUCTION_STOCK_IN posts the entered actual for Crispy Chicken Piece at the outlet
```

Scenario: Cooked ahead — confirm before the slot elapses

```
Given an In Progress task whose output is all cooked
When it is confirmed mid-slot
Then it moves to Done and posts its stock-in immediately
```

Scenario: An SFG prep task banks the intermediate

```
Given the Lunch window task for Mashed Potato (SFG) plans 6.000 grams
When it is confirmed at 6.000 g
Then PRODUCTION_STOCK_IN posts 6.000 g Mashed Potato at the outlet
And nothing deducts — its RM comes off later, at the exits' explosion
```

Story 2 — The SM inputs the actual cook via the full form (Phase 1)

User Story

As the SM, I confirm each production task by entering what was actually cooked — quantity, cook context, ready time, production code, and separate batches where cooked that way — so stock and plan-vs-actual reflect reality.

Acceptance Criteria

- Captures an actual quantity; plan shown for reference and editable — no confirm-at-plan tautology.
- Cycle-aware cook context: Hourly fryer+temperature; Daypart batch count; plus ready time and production code.
- A Daypart task defaults to one summed actual; the SM may add separate batch entries, each summing into the task's actual.
- Saving posts PRODUCTION_STOCK_IN immediately, idempotent on the entry id.
- This surface is the sole path in Phase 1 and remains available in Phase 2 — never retired.

AC Scenarios (Gherkin)

Scenario: SM confirms on plan

Given a task planned 27, shown as 27 (editable)
When the SM confirms 27 and saves
Then PRODUCTION_STOCK_IN posts 27 (a captured actual, not an assumed one)

Scenario: SM confirms a divergent quantity

Given a task shown at 50
When the SM enters 60 and saves
Then 60 posts, and the task shows planned 50 / actual 60 for plan-vs-actual

Scenario: SM records three batches for a daypart

Given a Breakfast Daypart task
When the SM records 20 @ 06:00, 30 @ 09:00, 50 @ 11:00 as three batch entries
Then three PRODUCTION_STOCK_IN post, each with its own ready time
And the task's actual reads 100
And each batch's shelf-life clock (EPIC 4) starts from its own produced_at

Scenario: A retried save does not double-post

Given the SM saves an entry and the request is retried
Then only one PRODUCTION_STOCK_IN exists for that entry

Story 3 — The crew see their station's work and confirm it fast

User Story

As a line cook, the tasks for my station appear as their time comes, and I confirm what I cooked in one tap, so recording never slows the line.

Acceptance Criteria

- The KDS renders only the signed-in station's released tasks, in slot order, in Pending / In Progress / Done.
- Tapping a ticket opens the capture sheet with identity and Production Code autofilled, Actual Cooked pre-filled with the plan.
- Confirming (Bump) advances to Done, ticks Order progress, posts PRODUCTION_STOCK_IN at the actual quantity.
- No stock figure or edit/void control appears anywhere on the KDS.
- An on-plan confirmation is achievable in a single tap after opening the sheet.

AC Scenarios (Gherkin)

Scenario: One-tap on-plan confirm

Given a Frying task (Crispy Chicken Piece, 15:00, planned 67) released and In Progress

When the cook opens it and taps Bump without changing the number
Then 67 posts as PRODUCTION_STOCK_IN and the ticket collapses to Done

Scenario: Divergent actual

Given the same task pre-filled at 67
When the cook edits it to 64 and Bumps
Then 64 posts and the task shows planned 67 / actual 64

Story 4 — Cycle-aware confirmation, on either surface

User Story

As the operator, a task asks me only what fits the item — pieces and fryer for Hourly, grams and batches for Daypart — so confirming takes seconds and can't be filled in wrong-shaped.

Acceptance Criteria

- Hourly captures pieces + fryer (A–D) + temperature + ready time; grams/batch fields never render.
- Daypart captures grams + batch count + ready time; fryer/temperature never render.
- The cycle is read from the plan line — never chosen at entry, on either surface.
- On the KDS specifically, Fresh Until derives read-only from ready time + the item's hold duration.

AC Scenarios (Gherkin)

Scenario: Hourly shape

Given Spicy Chicken (Personal) sits on an Hourly plan
When its 15:00 wave task is opened
Then the confirmation shows pieces, fryer, temperature, and ready time only

Scenario: Daypart shape

Given Berry Cooler premix sits on a Daypart plan
When its Lunch window task is opened
Then the confirmation shows grams, batch count, and ready time only

Story 5 — Correct or void an entry (back-office only)

User Story

As the SM/MOD, I fix a wrong entry the same day in place, or reverse an older one cleanly, so the ledger stays true without silent edits.

Acceptance Criteria

- Same-business-day: an authorized role edits fields in place; stock-in adjusts; edit is stamped and retained for audit.
- Older, or full reversal: void writes a reversing PRODUCTION_VOID — stock-in only, both legs visible in audit.
- A void never writes a WASTE exit.
- Crew cannot edit or void anywhere, including the KDS — confirming only.
- Re-confirming after a void is a fresh entry.

AC Scenarios (Gherkin)

Scenario: Same-day in-place correction

Given a task was confirmed with the wrong quantity
When the SM edits it in place the same business day
Then the quantity updates, the stock-in adjusts, and the edit is stamped

Scenario: Void an older entry

Given yesterday's confirmation banked 10 batches instead of 1
When the SM voids it today
Then a reversing PRODUCTION_VOID posts and both movements remain in audit

Story 6 — Offline capture never double-counts (KDS)

User Story

As a crew member on flaky wifi, my Bumps still land, once each, with the right time.

Acceptance Criteria

- A Bump made offline queues locally and replays on reconnect.
- Replay posts each entry exactly once (idempotent on entry id).
- Each replayed entry preserves its produced_at — shelf-life clock starts from the cook moment, not the sync moment.

AC Scenarios (Gherkin)

Scenario: Replay after a drop

Given three Bumps made while offline
When the tablet reconnects
Then exactly three PRODUCTION_STOCK_IN post, each with its original produced_at

8. Dependencies

Dependency	Direction	Note
EPIC 2 exit-explosion engine + BOM	upstream	Exit-time explosion (sale + waste), yield formula — the outstanding computation reuses the same code path
The planning flow (Template-Based Production Planning)	sibling	Submit creates the Production Order and Tasks this confirms against; its Raw Material step consumes this spec's effectively-available figure
Production Order & Task Contract	upstream	Owens the Order/Task entities, state machine, and the per-station feed both surfaces render
Planning Template Definition	upstream	The plan line's cycle the confirmation shapes to; item hold duration feeds Fresh Until on the KDS
station_id master (other PM)	upstream	Station routing depends on it; reserved, not set in V1
qsr-inventory (per-shop stock)	downstream	The target of PRODUCTION_STOCK_IN
order-be publish path	upstream	The Production Order's transport to the floor
Dashflow notification path	upstream	EPIC 5's stock-status signal to the SM

Dependency	Direction	Note
Adjustment primitive (ADJUSTMENT trigger)	downstream gap	Doesn't exist yet — only entries older than the same-day window route here; lands with the Stock Management epic
EPIC 4 Waste Management	downstream	REWORK/DISPOSE job lines, expiry from produced_at/ready time, EOD SFG waste, actual-loss comparison
EPIC 5 DOI / stock status	downstream	Should consume effectively available; SM-side notifications
Dash POS platform	sibling	Host device, offline storage, sync transport for the KDS — POS-team owned

9. Risks & Decisions

Item	Type	Severity	Note
Reunified for portfolio	Editorial	—	Split into two PRDs on 03 Aug so the KDS's own richness (station routing, offline mechanics) could be spec'd separately; recombined here since both are one stock-in contract with two front ends, not competing specs
Actual capture — operator banks actual, not plan	Decided (PM)	—	The plan is a pre-filled, editable default, never an assumed post. Supersedes an earlier plan-as-executed position, which made plan-vs-actual a tautology
Multi-batch entry per task	Decided (PM)	Low	A task accepts 1..N batch entries; the task actual sums them, each batch keeping its own ready time/shelf-life clock
Two entry surfaces, both first-class in the end state	Decided (PM)	Medium	Phase 1: SM form is the sole path. Phase 2: KDS becomes primary; SM form stays available, never retired
Idempotency on the stock-in	Decided (PM)	Medium	One entry = one movement, keyed on entry id — a double-tapped or retried Save must not post twice
produced_at drives the shelf-life clock, not sync time	Decided (PM)	Medium	EPIC 4's expiry is computed from when the food was made, never when the record reached the server
SFG produced and stocked	Decided (stakeholder)	—	SFG is a plannable, producible, stocked cooked intermediate; supersedes an earlier “stores never produce SFG” stance
Stock gate dropped at entry; enforced at planning	Decided (PM)	Medium	The capture-then-hold gate is removed from confirmation; coverage intent is enforced upstream at planning. Raw may sit momentarily negative until a GR catches up — surfaced by EPIC 5
Capture crew-only on the KDS; correction manager-only in the back office	Decided (PM)	—	Segregation of duties, surface-enforced: the KDS has no correction UI

Item	Type	Severity	Note
Offline idempotency + produced_at preservation	Decided (PM)	Medium	Load-bearing on kitchen wifi — one entry = one movement, keyed on entry id; expiry from the cook moment
The three expiry fields on the SM form	! Open	Medium	End Shelf Life Qty / Expired Date / Reused Qty are EPIC 4 disposition data, not knowable at the cook. Until ruled: treated as EPIC-4 inputs
station_id reserved, master owned elsewhere	Assumption / cross-PM	Medium	Not set in V1 on either surface; falls back to a single/undifferentiated view
Outstanding estimate drifts on mid-shelf-life BOM edits	Risk	Low	Inherent to compute-on-read against the current BOM
Operators backdate produced_at inaccurately	Risk	Medium	Tolerance window ($\leq 3h$) limits blast radius
Role not enforced on confirm/correct in V1	Assumption	Medium	QSR role-management doesn't exist yet; misuse observable via audit
POS-team build timeline for the KDS	Assumption	Medium	Whether it lands in V1 is POS-gated; the back-office model doesn't depend on it to function